CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 6

Personal Bests and the Final Report

Starts from: SpeedrunCheckpoint5. **Ends at:** SpeedrunCheckpoint6. **Prepares you for:** Lab W14, Part 3: the comparator with a tie-breaker, and writing the report.

This checkpoint finishes the program. It fixes the second problem Checkpoint 4 found, NovaByte holding two of the three Hollow Knight places, by giving each player one *personal best* per board. It lists the world records newest first, using a `Comparator` built from lambdas. And it writes the whole report to a file, so the console is left for the log.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Personal bests

A player who submits many runs should not crowd other players off a board. Real leaderboards show each player's *personal best*: that player's fastest verified run on the board. So the plan is to find each player's best run first, and then let the leaderboard pick the top three of those.

Add this method to `SpeedrunReport`, below `main`, with an import for `java.util.Collection`:

```
private static Collection<Run> personalBests(List<Run> runs) {
    return runs.stream()
            .filter(Run::verified)
            .collect(Collectors.toMap(Run::player, run -> run,
                    (first, second) -> first.compareTo(second) <= 0 ? first : second))
            .values();
}
```

`Collectors.toMap` builds a map with one entry per key, and it takes three arguments:

- `Run::player`, the key: one entry per player.
- `run -> run`, the value: the run itself.
- The *merge rule*, a lambda that decides what to keep when two runs have the same key, that is, when a player has two runs. It keeps the one that comes first in natural order, the faster run. The expression `condition ? a : b` means "`a` if the condition is true, otherwise `b`."

When the map is built, `values()` returns its values, one run per player, as a `Collection<Run>`. The merge rule is required here. Without it, `toMap` refuses to keep two values for one key, and the

program fails on the first player with two runs. The message is shortened here:

```
Exception in thread "main" java.lang.IllegalStateException: Duplicate key Lumen (attempted
merging values Run[player=Lumen, ...] and Run[player=Lumen, ...])
```

Now use the method in the leaderboard loop. Replace the pipeline that fills `top`, and change the title:

```
Leaderboard<Run> top = new Leaderboard<>(3);
personalBests(entry.getValue()).forEach(top::add);
System.out.println();
printBoard(entry.getKey() + ": top 3 personal bests", top);
```

Run the program. The Hollow Knight board changes from the left to the right:

Checkpoint 5:

```
 1. NovaByte      33:34.8  2026-01-31
 2. NovaByte      34:36.7  2026-01-21
 3. Moss          38:16.5  2026-04-29
```

Personal bests:

```
 1. NovaByte      33:34.8  2026-01-31
 2. Moss          38:16.5  2026-04-29
 3. Vela          38:29.0  2026-05-21
```

NovaByte's slower run no longer counts, and Vela moves onto the board. The other five boards already had three different players, so they do not change.

Step 2: Background: Comparator

A class has one natural order, the one its `compareTo` defines. For any other order, Java uses a `Comparator`: an object whose `compare(a, b)` method follows the same sign rules as `compareTo`, but lives outside the class. Because `Comparator` is an interface with one method, a lambda can be one. This comparator puts runs in date order, newest first:

```
(a, b) -> b.date().compareTo(a.date())
```

Swapping `a` and `b` on the right reverses the order, which is easy to get wrong and hard to read. The `Comparator` interface has methods that build the same thing step by step:

```
Comparator.comparing(Run::date)                 // oldest first
Comparator.comparing(Run::date).reversed()      // newest first

// newest first, with ties broken by natural order:
Comparator.comparing(Run::date).reversed()
        .thenComparing(Comparator.naturalOrder())
```

`comparing` takes the value to compare by, `reversed` flips the order, and `thenComparing` adds a tie-breaker that is used only when the first comparison is a tie.

One trap: write `comparing` with a method reference, as above. With a lambda, `Comparator.comparing(run -> run.date()).reversed()` does not compile, because Java cannot work out the lambda's parameter type before `reversed` is applied:

```
error: cannot find symbol
  symbol:   method date()
```

If you need a lambda there, give the parameter its type: `(Run run) -> run.date()`.

Step 3: World records, newest first

A visitor to a leaderboard site wants to see the newest records first. Add this constant at the top of `SpeedrunReport`, below `LOG`:

```
private static final Comparator<Run> NEWEST_FIRST =
        Comparator.comparing(Run::date).reversed()
                .thenComparing(Comparator.naturalOrder());
```

A comparator is an object like any other, so it can be stored in a constant, given a name that says what it does, and reused. Now replace the World Records loop from Checkpoint 5 with a loop that collects the records into a list, then sorts and prints it:

```
List<Run> records = new ArrayList<>();
for (List<Run> boardRuns : runsByBoard.values()) {
    boardRuns.stream()
             .filter(Run::verified)
             .min(Comparator.naturalOrder())
             .ifPresent(records::add);
}
records.sort(NEWEST_FIRST);

System.out.println();
System.out.println("World Records, newest first");
for (Run run : records) {
    System.out.printf("  %s  %-43s %-10s %10s%n", run.date(), run.board(),
            run.player(), RunTime.format(run.time()));
}
```

`ifPresent` is another way to use an `Optional`: it runs its lambda only when there is a value, so a board with no verified runs is simply left out. `records.sort(NEWEST_FIRST)` sorts the list in place with the comparator. Run the program:

```
World Records, newest first
  2026-09-13  The Legend of Zelda: Ocarina of Time, 100%  Sprocket    3:54:51.2
  2026-08-14  Portal, Inbounds                            Kestrel       10:20.1
  2026-02-14  Celeste, Any%                               Kestrel       26:41.6
  2026-01-31  Hollow Knight, Any%                         NovaByte      33:34.8
  2026-01-16  Super Mario 64, 16 Star                     Rook          15:54.4
  2025-12-28  Super Mario 64, 120 Star                    Sprocket    1:38:42.1
```

None of these records share a date, so the tie-breaker is never used with this data, but it keeps the order predictable if two records ever do.

Step 4: Write the report to a file

A report belongs in a file, where it can be saved and shared, and the console should be left for the log. A `PrintWriter` has the same `println` and `printf` methods as `System.out`, so moving the report is mostly a matter of changing where the lines go. Organize the report into one method per section, each taking the `PrintWriter` as a parameter:

```
private static void writeReport(String outFile, List<Run> runs) throws IOException {
    Map<Board, List<Run>> runsByBoard = runs.stream()
            .collect(Collectors.groupingBy(Run::board, TreeMap::new,
                    Collectors.toList()));
    try (PrintWriter out = new PrintWriter(outFile)) {
        writeSummary(out, runs);
        writeBoards(out, runsByBoard);
        writeRecords(out, runsByBoard);
    }
}
```

Then move the code that is now in `main` into three section methods, with these headers:

```
private static void writeSummary(PrintWriter out, List<Run> runs)
private static void writeBoards(PrintWriter out, Map<Board, List<Run>> runsByBoard)
private static void writeRecords(PrintWriter out, Map<Board, List<Run>> runsByBoard)
```

- `writeSummary` starts by printing a title line, `Speedrun Leaderboard Report`, then gets the verified count, the players, and runs per game.
- `writeBoards` gets the leaderboard loop.
- `writeRecords` gets the World Records code from Step 3.

In all three, and in `printBoard`, change every `System.out` to `out`, and give `printBoard` a `PrintWriter out` as its first parameter. Finally, in `main`, replace everything after the `"Loaded ..."` log line with:

```
writeReport(reportName, runs);
LOG.info("Wrote " + reportName);
```

and add `String reportName = "files/leaderboard_report.txt";` next to `fileName`. Run the program. The console now shows only the log, ending with:

```
INFO: Loaded 41 runs and rejected 9 lines
INFO: Wrote files/leaderboard_report.txt
```

Refresh the `files` folder with F5 and open `leaderboard_report.txt`:

```
Speedrun Leaderboard Report
41 runs, 35 verified
10 players:
[GrapeSoda, Kestrel, Lumen, Moss, NovaByte, PixelPilot, Rook, Sprocket, Tangent, Vela]

Runs per game
```

```
  Celeste                                 8
  Hollow Knight                           7
  Portal                                  7
  Super Mario 64                         13
  The Legend of Zelda: Ocarina of Time    6

Celeste, Any%: top 3 personal bests
 1. Kestrel       26:41.6  2026-02-14
 2. Moss          26:41.6  2026-05-03
 3. Vela          27:17.7  2026-05-20

Hollow Knight, Any%: top 3 personal bests
 1. NovaByte      33:34.8  2026-01-31
 2. Moss          38:16.5  2026-04-29
 3. Vela          38:29.0  2026-05-21

Portal, Inbounds: top 3 personal bests
 1. Kestrel       10:20.1  2026-08-14
 2. PixelPilot    10:30.1  2026-02-05
 3. Rook          10:52.5  2026-02-08

Super Mario 64, 120 Star: top 3 personal bests
 1. Sprocket    1:38:42.1  2025-12-28
 2. PixelPilot  1:41:04.5  2026-07-26
 3. Moss        1:41:26.1  2026-10-13

Super Mario 64, 16 Star: top 3 personal bests
 1. Rook          15:54.4  2026-01-16
 2. Moss          16:22.8  2026-09-05
 3. Lumen         17:03.4  2025-11-15

The Legend of Zelda: Ocarina of Time, 100%: top 3 personal bests
 1. Sprocket    3:54:51.2  2026-09-13
 2. Lumen       4:00:30.8  2026-06-16
 3. Moss        4:14:17.6  2026-03-17

World Records, newest first
  2026-09-13  The Legend of Zelda: Ocarina of Time, 100%  Sprocket    3:54:51.2
  2026-08-14  Portal, Inbounds                            Kestrel       10:20.1
  2026-02-14  Celeste, Any%                               Kestrel       26:41.6
  2026-01-31  Hollow Knight, Any%                         NovaByte      33:34.8
  2026-01-16  Super Mario 64, 16 Star                     Rook          15:54.4
  2025-12-28  Super Mario 64, 120 Star                    Sprocket    1:38:42.1
```

Your project should now match SpeedrunCheckpoint6, the finished program.

What you built, and where it goes next

Table 1 matches each checkpoint to the part of Lab W14 that asks for the same thing. The lab uses different data, different rules, and a different generic class, so the code will not carry over, but the steps will.

Table 1: Each checkpoint, and the part of Lab W14 that repeats it.

Checkpoint	What you built	Lab W14
1	Caught a missing file; a record that refuses bad values	Part 1
2	A checked exception, a parser that translates Java's exceptions, a loop that skips bad lines	Part 1
3	A natural order with `Comparable`; a generic class with a bounded type	Part 2
4	Collections chosen for the job, with no raw types	Part 2
5	Stream pipelines for counting, grouping, and finding the best	Part 3
6	A comparator with a tie-breaker; a report written to a file	Part 3

Check yourself

1. In `personalBests`, what would happen without the merge rule, and why?
2. Rewrite `first.compareTo(second) <= 0 ? first : second` as an `if` statement that returns the faster run.
3. What is the difference between `Run`'s `compareTo` and the `NEWEST_FIRST` comparator? Why can a class have only one of the first, but any number of the second?
4. What does `thenComparing(Comparator.naturalOrder())` add to `NEWEST_FIRST`, and when is it used?
5. Why is the report better in a file, with only the log on the console?

Answers

1. It would throw an `IllegalStateException` for a duplicate key as soon as it met a player with two verified runs on the same board, because `toMap` keeps one value per key and, without a merge rule, does not know which one to keep.
2. `if (first.compareTo(second) <= 0) \{ return first; \} else \{ return second; \}`
3. `compareTo` is written inside `Run` and defines its natural order, so there can be only one. A comparator is a separate object, so a program can have as many orders as it needs, such as fastest first for leaderboards and newest first for records.
4. A tie-breaker: when two records have the same date, they are ordered by `Run`'s natural order, fastest first. It is used only when the dates are equal.
5. A file can be saved, shared, and compared with earlier reports, and the report stays clean of log messages. The log, meant for the person running the program, stays on the console.